

Existence and Localization of a Limit Cycle in a Class of Benchmark Biomolecular Oscillators: Supplementary

Sidhanta Mohanty, Shaunak Sen

CONTENTS

I	Part - I: Complete Table III of Main Paper	2
II	Part - II: Interval-based Reachability Analysis	6
II-A	Problem-Statement	6
II-B	Methodology	7
II-C	Algorithms	7
II-C1	Algorithm 1 - Overall Classification Procedure (Main Loop)	8
II-C2	Algorithm 2 - classify	8
II-C3	Algorithm 3 - solve_ivp	9
II-C4	Algorithm 4 - nearest	10
II-C5	Algorithm 5 - classify_three_planes	11
II-C6	Algorithm 6 - multi_line_check	12
II-C7	Algorithm 7 - hull_line	13
II-D	Results and Discussions	14
II-E	Conclusion	16

The authors developed all algorithms presented in this supplementary. The LaTeX syntax and formatting were partially generated with the help of ChatGPT (OpenAI) as a support tool.

I. PART - I: COMPLETE TABLE III OF MAIN PAPER

The direction of flow of vector field within the smaller hyperrectangles for a 5-gene cyclic regulatory network is given below.

Table I: Direction of flow of vector field within the smaller boxes.

Box	Coordinates of the hyperrectangle	Direction of the flow of vector field
1	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 2 ↑ Box 32 ← Box 1 → Box 29 ↙ ↘ Box 31 Box 32
2	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 25 ↗ Box 2 → Box 26 ↘ Box 28
3	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 2 ↗ Box 3 → Box 18 ↘ Box 27
4	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 10 ↗ Box 4 → Box 3 ↘ Box 28
5	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 4 ↗ Box 5 → Box 17 ↘ Box 19
6	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 5 ↑ Box 10 ← Box 6 → Box 7 ↙ ↘ Box 9 Box 8
7	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 12 ↗ Box 7 → Box 13 ↘ Box 14
Continued on the next page.		

Continuation of Table I ...		
Box	Coordinates of the hyperrectangle	Direction of the flow of vector field
8	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 11 ↗ Box 8 → Box 16 ↘ Box 17
9	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 12 ↗ Box 9 → Box 15 ↘ Box 19
10	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 13 ↗ Box 10 → Box 16 ↘ Box 18
11	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 7 ↗ Box 11 → Box 21 ↘ Box 22
12	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 12 → Box 24
13	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 13 → Box 23
14	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 5 ↗ Box 14 → Box 24 ↘ Box 27
15	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 8 ↗ Box 15 → Box 20 ↘ Box 26
Continued on the next page.		

Continuation of Table I ...		
Box	Coordinates of the hyperrectangle	Direction of the flow of vector field
16	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 16 $\rightarrow$ Box 21
17	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 17 $\rightarrow$ Box 28
18	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 9 $\nearrow$ Box 15 $\rightarrow$ Box 23 $\searrow$ Box 25
19	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 19 $\rightarrow$ Box 26
20	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 11 $\nearrow$ Box 20 $\rightarrow$ Box 12 $\searrow$ Box 30
21	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 21 $\rightarrow$ Box 13
22	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 14 $\nearrow$ Box 22 $\rightarrow$ Box 17 $\searrow$ Box 31
23	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 23 $\rightarrow$ Box 12
Continued on the next page.		

Continuation of Table I ...		
Box	Coordinates of the hyperrectangle	Direction of the flow of vector field
24	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 24 $\rightarrow$ Box 19
25	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 15 $\nearrow$ Box 25 $\rightarrow$ Box 16 $\searrow$ Box 29
26	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 26 $\rightarrow$ Box 17
27	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 4 $\nearrow$ Box 27 $\rightarrow$ Box 13 $\searrow$ Box 32
28	$x_0 \leq x_1 \leq \frac{\alpha}{\gamma}$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 28 $\rightarrow$ Box 16
29	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $x_0 \leq x_5 \leq \frac{\alpha}{\gamma}$	Box 20 $\nearrow$ Box 29 $\rightarrow$ Box 21 $\searrow$ Box 23
30	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $0 \leq x_3 \leq x_0$ $x_0 \leq x_4 \leq \frac{\alpha}{\gamma}$ $0 \leq x_5 \leq x_0$	Box 22 $\nearrow$ Box 30 $\rightarrow$ Box 24 $\searrow$ Box 26
31	$0 \leq x_1 \leq x_0$ $0 \leq x_2 \leq x_0$ $x_0 \leq x_3 \leq \frac{\alpha}{\gamma}$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 21 $\nearrow$ Box 31 $\rightarrow$ Box 27 $\searrow$ Box 28
Continued on the next page.		

Continuation of Table I ...		
Box	Coordinates of the hyperrectangle	Direction of the flow of vector field
32	$0 \leq x_1 \leq x_0$ $x_0 \leq x_2 \leq \frac{\alpha}{\gamma}$ $0 \leq x_3 \leq x_0$ $0 \leq x_4 \leq x_0$ $0 \leq x_5 \leq x_0$	Box 3 ↗ Box 32 → Box 23 ↘ Box 24

II. PART - II: INTERVAL-BASED REACHABILITY ANALYSIS

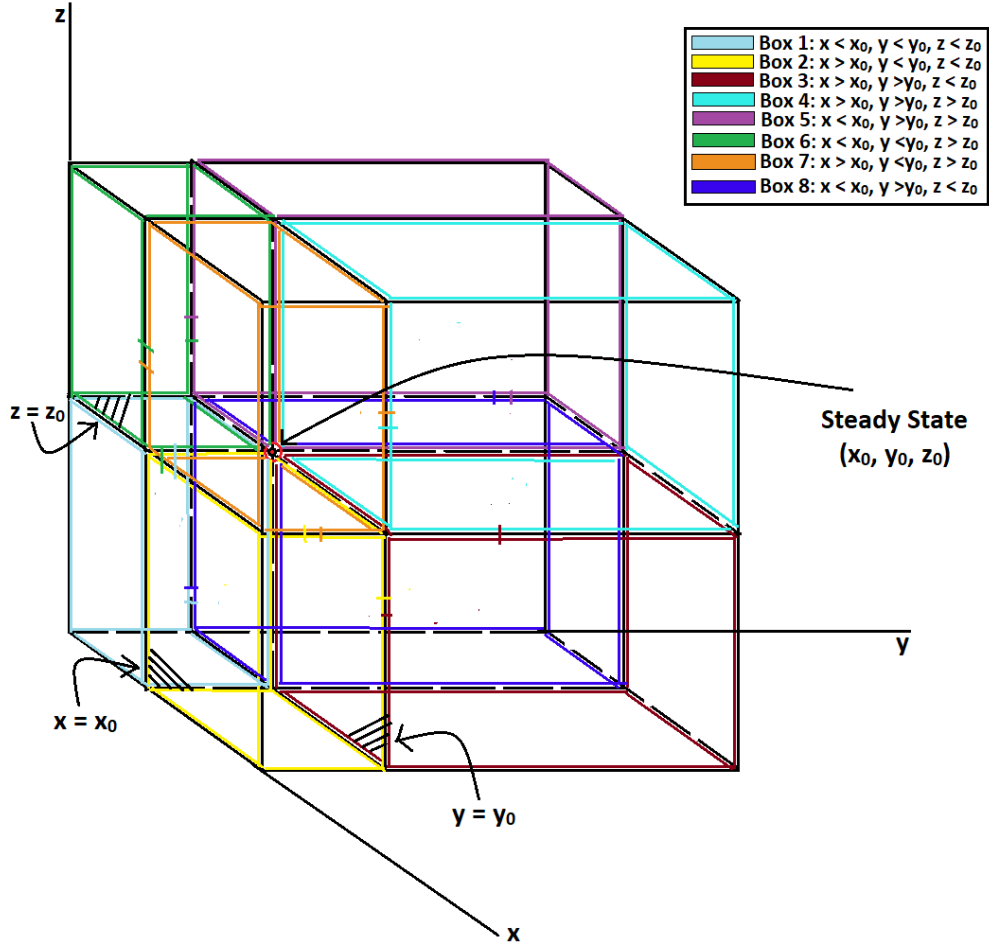


Figure 1: The cube is divided into 8 smaller boxes by the planes $x = x_0$, $y = y_0$ and $z = z_0$.

Using the direction of the trajectories obtained from the signs of $\dot{x}$, $\dot{y}$ and $\dot{z}$, we get a continuous map of a cross-section onto itself. The map is as follows: $2 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 2$.

A. Problem-Statement

The objective is to extend the core idea of the Brouwer's Fixed Point theorem to an interval-based Reachability Analysis and localize the limit cycle trajectory. We aim to check whether a specified

hyperrectangular region of initial conditions evolves in a manner that the system trajectory either returns to and overlaps with or entirely traverses the original region.

B. Methodology

The methodology involves dividing a search space into smaller hyperrectangles, evolving the system dynamics from each via interval-based Reachability Analysis (flowpipes), and examining the return map to classify each region as: (i) Completely contains the limit cycle, (ii) Partially contains the limit cycle, or (iii) Does not contain the limit cycle. This classification fundamentally depends on a geometric subset test to check if the computed return set, X_t , is entirely encompassed within the original set, X_0 .

To reduce the computational complexity of performing this subset test on 3D zonotopic sets, our approach projects the sets onto the three principal 2D planes (xy , yz , and xz). For each projection, a thorough `multi_line_check` is performed, which evaluates the 2D subset property by traversing directional 1D “slices” across the sets. The classification is thereafter done based on the outcomes of these evaluations:

- **Contains limit cycle completely** is assigned solely when an intersection is observed and all three 2D projection evaluations unanimously indicate “complete”.
- **Does not contain limit cycle** is assigned when a failure is observed. This arises if (i) at least one 2D projection verification exceeds the subset condition, or (ii) no observable intersection is identified and the bounding box of the return set does not intersect with that of the original set.
- **Partially contains limit cycle** is utilized in uncertain scenarios. This occurs if (i) an observable intersection is observed but the three-plane evaluation produces mixed outcomes (a combination of “complete” and “uncertain”), or (ii) no apparent intersection is recognized yet the resultant set represents a “near miss,” characterized by an overlap of its bounding box with that of the original set.
- **Needs further check** is assigned for purely computational failures, such as when the `nearest` function cannot identify a valid return candidate.

C. Algorithms

An external loop initially calls the function `divide`, to split the entire search space into smaller subsets. The `classify` function is thereafter applied iteratively to each subset, facilitating systematic identification and categorization and thereby localization of regions in the phase space that contain the limit cycle of the repressilator model.

The main function, `classify`, executes the core analysis for each initial set. The operation depends on several subroutines: (i) `solve_ivp`, which simulates the system’s dynamics, where the initial conditions are defined as interval sets generated by partitioning the phase plane into a grid via the `divide`; (ii) `nearest`, which determines the trajectory segment closest to the initial set; (iii) `classify_three_planes`, which enhances classification via geometric containment checks across various 2D projections; and (iv) `multi_line_check` and `hull_line`, which ensure accurate verification of containment boundaries.

The algorithms outlined here define the process for identifying and categorizing and thereby localizing limit cycle trajectory within the phase space of the repressilator.

1) *Algorithm 1 - Overall Classification Procedure (Main Loop)*: This algorithm begins by dividing the entire search space into a grid of smaller, distinct boxes. The algorithm subsequently checks each of these boxes, executing the primary classification process to determine its correlation with the system's limit cycle. According to the returning set, each box is classified into one of the three categories: total containment, partial containment, or requiring further investigation due to computational uncertainty. Boxes found not to contain the cycle are eliminated, so significantly reducing the search space.

Algorithm 1: Overall Classification Procedure (Main Loop)

Input: Search space X_{search} , number of divisions n_{div}

Output: Lists: *Limit_Cycle*, *idx_limit*, *Limit_Cycle_partial*, *idx_partial*, *Further_check*, *idx_further_check*, *Failed_Boxes*

▷ Initialize output lists.

```

1 Limit_Cycle ← [ ], idx_limit ← [ ], Limit_Cycle_partial ← [ ], idx_partial ← [ ],
  Further_check ← [ ], idx_further_check ← [ ], Failed_Boxes ← [ ]

2  $S \leftarrow \text{divide}(X_{search}, n_{div})$                                 ▷ Split the initial search space.
3 foreach ( $X_0, idx$ )  $\in \text{enumerate}(S)$  do
4   status ← classify( $X_0, idx$ )
5   if status = “Contains limit cycle completely” then
6     append  $X_0$  to Limit_Cycle
7     append idx to idx_limit
8   else if status = “Contains limit cycle partially” then
9     append  $X_0$  to Limit_Cycle_partial
10    append idx to idx_partial
11  else if status = “Needs further check with stricter settings” then
12    append  $X_0$  to Further_check
13    append idx to idx_further_check

    ▷ Boxes that do not contain the cycle are discarded.
    ▷ Failed boxes are added to Failed_Boxes inside solve_ivp.
```

2) *Algorithm 2 - classify*: This algorithm functions as the primary classification function for a singular initial set (X_0). Initially, it models the system's trajectory via the *solve_ivp* function. It then determines the point of closest return to X_0 utilizing the *nearest* function. The categorization proceeds according to the characteristics of the return set. If a valid geometric intersection is found, a rigorous multi-plane geometric verification is carried out. If just a “closest” return point is identified without an intersection, the algorithm employs a more cautious check. This check is done by evaluating

the overlap of the bounding boxes of the initial and final sets. This approach involves error handling and guarantees reliable classification.

Algorithm 2: Main routine: `classify`

Input: Initial set X_0 (Hyperrectangle), box index idx , skip parameter s , directions N

Output: Classification string

```

1 try ( $\mathcal{S}, success$ )  $\leftarrow solve\_ivp(X_0)$ 
2 if not success then
3   return “solve_ivp failed”
4   append  $X_0$  to Failed_Boxes
5 ( $k^*, status$ )  $\leftarrow nearest(\mathcal{S}, s)$ 
6 if  $status = :failed$  then
7   return “Needs further check with stricter settings”
8  $X_t \leftarrow set(\mathcal{S}[k^*])$ 
9  $X_t^{HR} \leftarrow overapproximate(X_t, Hyperrectangle)$ 
10 if  $status = :intersected$  then
11   return classify_three_planes( $X_0, X_t^{HR}, N$ )
12 else if  $status = :closest\_fallback$  then
13   if  $intersection(X_0, X_t^{HR}) \neq \emptyset$  then
14     return “Contains limit cycle partially”
15   else
16     return “Does not contain limit cycle”
17 catch any error return “Needs further check with stricter settings”

```

3) *Algorithm 3 - solve_ivp*: This algorithm numerically solves the initial value problem (IVP) for a specified initial set, X_0 . It employs the Taylor-model-based solver, `TMJets21a`, to calculate the evolution of the system’s dynamics over a predetermined time horizon. The resultant solution is contained within a series of zonotopes, creating a “flowpipe” that ensures the true trajectory remains contained. In case the solver encounters trouble with a specific initial set, the function records the set for subsequent examination and issues a failure flag, so allowing the entire analysis to continue uninterrupted.

Algorithm 3: `solve_ivp`

Input: Initial set X_0

Output: Solution flowpipe $\mathcal{S}$, success flag

```

1 try Define IVP with repressilator dynamics from  $X_0$ 
2 Solve using TMJets21a algorithm
3  $\mathcal{S} \leftarrow \text{overapproximate}(\text{solution}, \text{Zonotope})$ 
4 return ( $\mathcal{S}, \text{true}$ )
5 catch failure append  $X_0$  to Failed_Boxes
6 return (nothing, false)

```

4) *Algorithm 4 - nearest*: This algorithm identifies the system’s trajectory “return time” by examining the solution flowpipe. It utilizes a two-step approach to guarantee both precision and robustness. In the initial phase, it examines the time steps of the solution, finding the earliest geometric intersection with the beginning state. If an intersection is found, it provides a successful result. Simultaneously, it monitors the state that is geometrically nearest to the starting state, as determined by the distance between their centers. A skip parameter s is used to avoid the categorization of the initial time steps in the solution flowpipe as the nearest return set. If the initial step fails to identify a direct intersection, a more focused search is used within a limited region surrounding the nearest option. Should this also fail, the method defaults to returning the index of the “closest” state, so offering a heuristic-based outcome in the absence of a confirmed one. This dual methodology guarantees the identification of a significant return point, even in geometrically complex or numerically complex situations.

Algorithm 4: nearest

Input: Solution flowpipe $\mathcal{S}$, skip parameter s **Output:** Tuple (k^*, status)

```

1  $Z_0 \leftarrow \text{set}(\mathcal{S}[1]), c_0 \leftarrow \text{center}(Z_0)$ 
2  $\text{best\_dist} \leftarrow \infty, \text{best\_k} \leftarrow \text{nothing}$ 

   $\triangleright$  Step 1: Search for an intersection.
3 for  $k = s$  to  $\text{enumerate}(\mathcal{S})$  do
4    $Z_k \leftarrow \text{set}(\mathcal{S}[k])$ 
5   try if  $\text{intersection}(Z_k, Z_0) \neq \emptyset$  then
6     return  $(k, : \text{intersected})$ 
7   catch error  $\triangleright$  Ignore intersection check errors.
8    $d \leftarrow \|\text{center}(Z_k) - c_0\|$ 
9   if  $d < \text{best\_dist}$  then
10     $\text{best\_dist} \leftarrow d, \text{best\_k} \leftarrow k$ 

   $\triangleright$  Step 2: Fallback if no intersection was found.
11 if  $\text{best\_k} \neq \text{nothing}$  then
12    $\triangleright$  Search in a small window around the closest candidate.
13    $\text{window} \leftarrow \max(1, \text{best\_k} - 5) \dots \min(\text{enumerate}(\mathcal{S}), \text{best\_k} + 5)$ 
14   foreach  $\tilde{k} \in \text{window}$  do
15     if  $\text{intersection}(\text{set}(\mathcal{S}[\tilde{k}]), Z_0) \neq \emptyset$  then
16       return  $(\tilde{k}, : \text{intersected})$ 
17   return  $(\text{best\_k}, : \text{closest\_fallback})$ 
18 return  $(1, : \text{failed})$ 

```

5) *Algorithm 5 - classify_three_planes*: This algorithm confirms whether a 3D return set (X_t) is contained within its matching 3D initial set (X_0). The algorithm simplifies the problem by projecting both hyperrectangles onto the three primary 2D coordinate planes: xy , yz , and xz , rather than executing a complex 3D comparison. For each of these two-dimensional projections, a containment check is conducted with the `multi_line_check` function. The final classification of “completely contained”, “partially contained”, or “not contained” is made based on the cumulative outcomes from the three planes, guaranteeing a dependable conclusion even when one of the 2D assessments may be uncertain.

Algorithm 5: `classify_three_planes`

Input: Sets X_0, X_t (Hyperrectangles), directions N
Output: Classification

```

1 complete  $\leftarrow$  0, not_complete  $\leftarrow$  0, uncertain  $\leftarrow$  0
2 foreach plane  $(i, j) \in \{(1, 2), (2, 3), (1, 3)\}$  do
3    $X_0^p \leftarrow \text{project}(X_0, [i, j])$ ,  $X_t^p \leftarrow \text{project}(X_t, [i, j])$ 
4   if  $\| \text{center}(X_0^p) - \text{center}(X_t^p) \| < \text{tolerance}$  then
5     complete  $\leftarrow$  complete + 1
6     continue
7   result  $\leftarrow$  multi_line_check( $X_0^p, X_t^p, N$ )
8   if result = :complete then
9     complete  $\leftarrow$  complete + 1
10  else if result = :not_complete then
11    not_complete  $\leftarrow$  not_complete + 1
12  else
13    uncertain  $\leftarrow$  uncertain + 1

  ▷ Final decision based on results from the three planes.
14 if not_complete > 0 then
15   return “Does not contain limit cycle”
16 else if complete == 3 then
17   return “Contains limit cycle completely”
18 else
19   return “Contains limit cycle partially”

```

6) *Algorithm 6 - multi_line_check*: This algorithm conducts a containment check between two 2D sets by probing them from multiple directions. For each of the N chosen directions, it first performs an exact geometric check by comparing the line segments corresponding to the intersections of the two sets along that direction. If the primary geometric test fails because of numerical instability or nonconvex boundary behavior, the algorithm switches to a more robust backup method that uses support functions to estimate the maximum extent of each set along the same direction.

The support function, denoted by $\rho(u, X)$, gives the farthest distance of set X in the direction of the unit vector u . By comparing $\rho(u, X_t^p)$ and $\rho(u, X_0^p)$ and their opposites, $\rho(-u, X_t^p)$ and $\rho(-u, X_0^p)$, the algorithm can determine whether the evolved set X_t^p extends beyond the initial set X_0^p along any direction u . If this occurs, geometric containment is refuted. This methodology guarantees reliability: if any direction conclusively shows that X_t^p exceeds X_0^p , containment is immediately rejected. If all tests pass, containment is considered complete. In case any numerical errors occur, the result is conservatively

marked as uncertain, avoiding false positives.

Algorithm 6: multi_line_check

Input: 2D projections X_0^p, X_t^p , number of directions N

Output: Containment status symbol

```

1 Generate  $N$  unit directions  $\{u_k\}$ 
2  $fail\_count \leftarrow 0$ ,  $uncertain \leftarrow false$ 
3 foreach  $u_k$  in directions do
4    $success \leftarrow false$ 
   ▷ Primary Method: Geometric check.
5   try  $l \leftarrow hull\_line(X_0^p, u_k)$ 
6    $inter_0 \leftarrow l \cap X_0^p$ ,  $inter_t \leftarrow l \cap X_t^p$ 
7   if  $inter_t \not\subseteq inter_0$  then
8     return :not_complete
9    $success \leftarrow true$ 
10  catch error  $fail\_count \leftarrow fail\_count + 1$ 
   ▷ Fallback Method: Support function check.
11  if not success then
12    try if  $\rho(u_k, X_t^p) > \rho(u_k, X_0^p)$  or  $\rho(-u_k, X_t^p) > \rho(-u_k, X_0^p)$  then
13      return :not_complete
14    catch error  $uncertain \leftarrow true$ 
15 if  $fail\_count/N > max\_fail\_fraction$  or  $uncertain$  then
16   return :uncertain
17 else
18   return :complete

```

7) *Algorithm 7 - hull_line*: This algorithm calculates a line segment that indicates the overall extent, or “width” of a 2D geometric set in a designated direction. It achieves this by identifying the two most extreme points on the boundary of the set utilizing the support function σ , which determines the points that are farthest in the specified direction and its opposite. The function includes a fallback mechanism to guarantee robustness against numerical complexities or complex geometries. Should the support function fail on the original set, it then calculates a more straightforward hyperrectangle enclosure (a bounding box) and subsequently identifies the extreme points on that simplified shape. This ensures that a valid line segment is consistently produced.

Algorithm 7: hull_line

Input: 2D set Z , direction u **Output:** Line segment connecting extreme points

```

1  $d \leftarrow u/\|u\|$ 
2 try  $p \leftarrow \sigma(d, Z)$  ▷ Support point in direction  $d$ .
3  $q \leftarrow \sigma(-d, Z)$  ▷ Support point in direction  $-d$ .
4 return LineSegment( $p, q$ )
5 catch error ▷ Fallback to a bounding box if support vector fails.
6  $HR \leftarrow \text{overapproximate}(Z, \text{Hyperrectangle})$ 
7  $p \leftarrow \sigma(d, HR)$ 
8  $q \leftarrow \sigma(-d, HR)$ 
9 return LineSegment( $p, q$ )
```

D. Results and Discussions

The above interval-based Reachability Analysis is used to classify the cross-section between boxes 2 and 7. This framework used here successfully divides the above cross-section of the repressilator according to their underlying dynamical behavior. The methodology, based on partitioning the phase space and analyzing the flowpipe of each partition, yielded various findings, which are visually summarized in Figure 2.

The analysis classified the vast majority of the partitioned region of this cross-section as not containing a periodic orbit. Region a , `small_X0[1191]` in Figure 2, contains the unstable steady state (x_0, x_0, x_0) (the value of x_0 for this simulation is 5.361) of the system and is excluded from the Reachability Analysis. Region b , `small_X0[1121, 1156]`, shows the classification of boxes near the excluded steady-state point as “Completely Contains Cycle”. This result is physically counterintuitive, as these trajectories lie deep within the steady-state basin of attraction and should converge to it. For trajectories initialized in the basin of the steady state, the reachable sets contract monotonically as they approach equilibrium. Consequently, the final contracted overapproximation can become geometrically contained within the initial, larger hyperrectangle, satisfying the inclusion criterion despite the absence of periodic behaviour. A distinct cluster of initial sets Region c , `small_X0[793:795, 828:830, 865]`, in Figure 2, was classified as “Completely Contains Cycle” or “Partially Contains Cycle”. This region is located far from the system’s unstable steady state which is contained in the Region a . Further in Figure 3 the numerical solution of the repressilator model is plotted along with the surface between boxes 2 and 7. Here the trajectory passes through the Region c in Figure 2, which was classified by the algorithm as containing the limit cycle completely or partially. This finding validates the method’s efficacy in localizing the region in which the limit cycle exists.

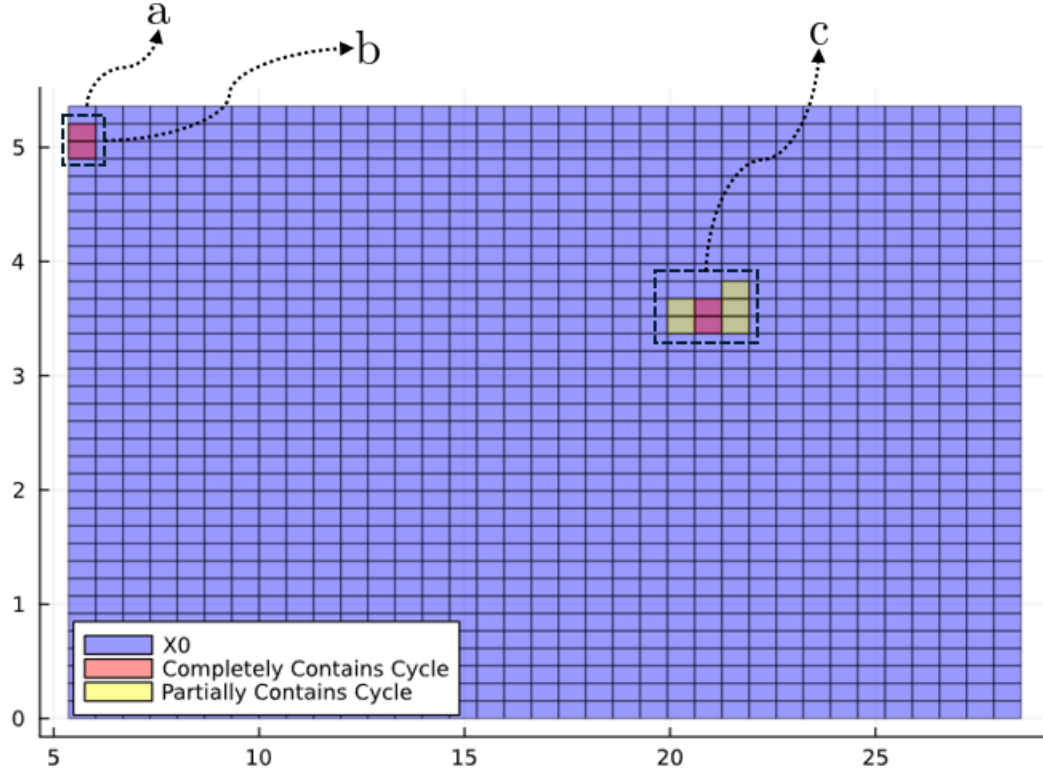


Figure 2: xy -projection of the surface between boxes 2 and 7 showing the geometric classification results. Region a (`small_X0[1191]`) contains the steady-state point and was excluded from the reachability analysis. Region b (`small_X0[1121, 1156]`) shows a false “Completely Contains Cycle” classification arising from convergence near the steady state. Region c (`small_X0[793:795, 828:830, 865]`) corresponds to the cluster of boxes through which the limit cycle passes completely or partially.

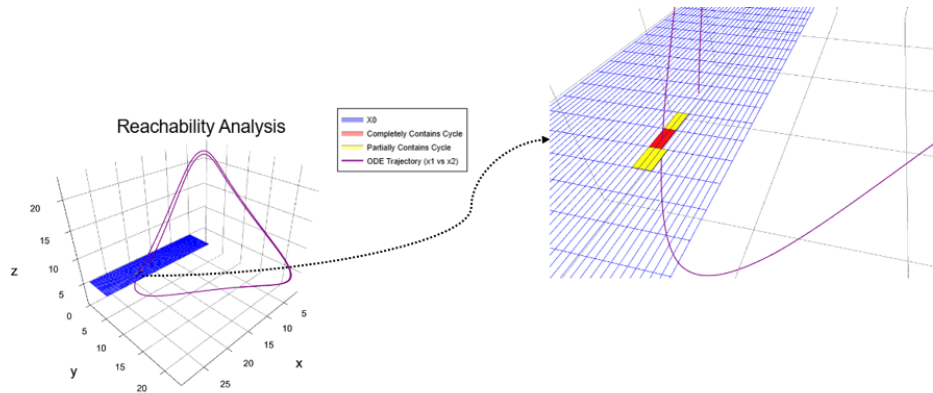


Figure 3: 3D plot of the numerical solution of the repressilator model. The resulting trajectory passes through the Region c in Figure 2

E. Conclusion

In summary, the rigorous localization of the region of existence of the limit cycle in the repressilator relies on an interval-based Reachability Analysis, which, in conjunction with Brouwer's Fixed Point theorem, can provide a significantly tighter enclosure of the limit cycle trajectory. Also interpreting the false *complete* classifications near the steady state (Region *b*) due to contraction was essential for validating the final results.